Transformacje Model Informacyjny (PIM) - Implementacja w Java + JPA (Hibernate) (PSM)

- Wstęp
- Odwzorowania atrybutów typów prostych
  - PIM
  - PSM - implementacja
    - Plik: ClassA.java
    - Plik: DataTypeA.java
    - Plik: EnumerationA.java
- Odwzorowania asocjacji 1-1
  - Przykład 1
    - PIM
    - PSM - Implementacja
      - Plik: ClassA.java
      - Plik: ClassB.java
  - Przykład 2
    - PIM
    - PSM - Implementacja
      - Plik: ClassA1.java
      - Plik: ClassB1.java
- Odwzorowania asocjacji 1-*
  - Przykład 1
    - PIM
    - PSM - Implementacja
      - Plik: ClassA2.java
      - Plik: ClassB2.java
  - Przykład 2
    - PIM
    - PSM - Implementacja
      - Plik: ClassA3.java
      - Plik: ClassB3.java
  - Przykład 3
    - PIM
    - PSM - Implementacja
      - Plik: ClassA4.java
      - Plik: ClassB4.java
- Odwzorowania asocjacji *-*
  - Przykład 1

- PIM
- PSM - Implementacja
  - Plik: ClassA5.java
  - Plik: ClassB5.java

Odwzorowania asocjacji z kwalifikatorem
- Przykład 1
  - PIM
  - PIM - odwzorowanie
- Przykład 2
  - PIM
  - PIM - odwzorowanie

Odwzorowania klas asocjacyjnych 1-1
- Przykład 1
  - PIM
  - PIM - odwzorowanie
- Przykład 2
  - PIM
  - PIM - odwzorowanie
- Przykład 3
  - PIM
  - PIM - odwzorowanie

Odwzorowania klas asocjacyjnych 1-*
- Przykład 1
  - PIM
  - PIM - odwzorowanie
- Przykład 2
  - PIM
  - PIM - odwzorowanie

Odwzorowania klas asocjacyjnych *-*
- Przykład 1
  - PIM
  - PIM - odwzorowanie
- Przykład 2
  - PIM
  - PIM - odwzorowanie

Odwzorowania relacji generalizacji.
- Przykład 1
  - PIM
  - PSM - Implementacja
    - Plik: ClassD.java
    - Plik: ClassD1.java
- Przykład 2
  - PIM
  - PSM - Implementacja

Plik: AbstractClassA.java
Plik: ClassA.java
Plik: ClassB.java
Przykład 3
PIM
PSM - Implementacja
Plik: GeneralClassA2.java
Plik: ClassA2.java
Plik: ClassB2.java
Odwzorowania relacji generalizacji. Wielodziedziczenia, wieloinstancyjność.
Przykład 1
PIM
PIM - odwzorowanie
Przykład 2
PIM
PIM - odwzorowanie
Przykład 3
PIM
PIM - odwzorowanie

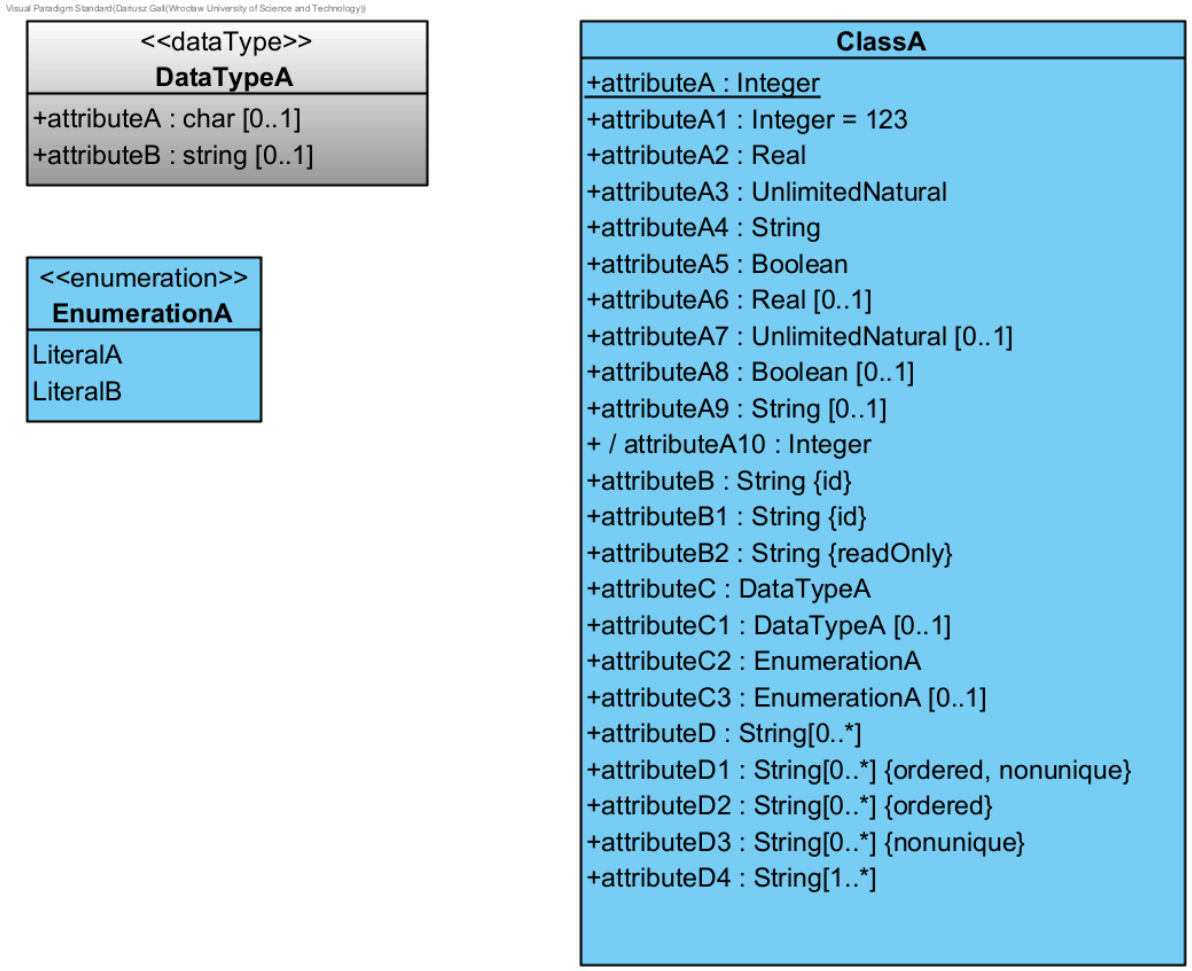
Wstęp

Dokument zawiera ilustracje odwzorowań wybranych konstrukcji analitycznych występujących na modelu informacyjnym (diagram klas) do konstrukcji na poziomie implementacji, implementacji w języku Java z wykorzystaniem biblioteki Hibernate.

Odwzorowania atrybutów typów prostych

Przedstawiono odwzorowania klasy, która zawiera atrybuty typów strukturalnych, prymitywów, enumeratorów. W dalszej części znajdują się przykłady konstrukcji analitycznych i ich odwzorowanie do kodu w Javie + Hibernate.

PIM



PSM - implementacja

Plik: ClassA.java

```
package pl.edu.pwr.abis.domain.basic;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotNull;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import lombok.AccessLevel;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "basic_classa")
@Getter
@Setter
@Table(uniqueConstraints = { @UniqueConstraint(columnNames = { "attributeB",
```

```
"attributeB1" }) })
public class ClassA {

   @Id
   private Long id;

   private static int attributeA;

   @Basic
   private int attributeA1 = 123; //long

   @Basic
   private double attributeA2; // float

   @Basic
   private long attributeA3;

   @NotNull
   @Basic
   private String attributeA4;

   @Basic
   private boolean attributeA5;

   @Basic
   private Double attributeA6; // Float, BigDecimal

   @Basic
   private Long attributeA7; // BigDecimal

   @Basic
   private Boolean attributeA8;

   @Basic
   private String attributeA9;

   @Transient
   public int getAttributeA10() {
       return 0;
   }

   @NotNull
   @Basic
   private String attributeB; // gdy jeden atrybut {id} to @Column(unique = true)

   @NotNull
   @Basic
   private String attributeB1; // gdy jeden atrybut {id} to @Column(unique = true)

   @NotNull
   @Basic
   @Setter(AccessLevel.NONE)
   private String attributeB2;

   @NotNull
```

```
    @Embedded
    @AttributeOverrides(
        {
            @AttributeOverride(name = "attributeA", column = @Column(name =
"attributeC_attributeA")),
            @AttributeOverride(name = "attributeB", column = @Column(name =
"attributeC_attributeB")),
        }
    )
    private DataTypeA attributeC;

    @Embedded
    @AttributeOverrides(
        {
            @AttributeOverride(name = "attributeA", column = @Column(name =
"attributeC1_attributeA")),
            @AttributeOverride(name = "attributeB", column = @Column(name =
"attributeC1_attributeB")),
        }
    )
    private DataTypeA attributeC1;

    @NotNull
    @Enumerated
    private EnumerationA attributeC2;

    @Enumerated
    private EnumerationA attributeC3;

    @ElementCollection(targetClass = String.class, fetch = FetchType.EAGER)
    @CollectionTable(name = "attributeds", joinColumns = @JoinColumn(name =
"classa_id"))
    @Column(name = "attributed", nullable = false)
    private Set<String> attributeD = new HashSet<>();

    @ElementCollection(targetClass = String.class, fetch = FetchType.EAGER)
    @CollectionTable(name = "attributed1s", joinColumns = @JoinColumn(name =
"classa_id"))
    @Column(name = "attributed1", nullable = false)
    private List<String> attributeD1 = new ArrayList<>();

    @ElementCollection(targetClass = String.class, fetch = FetchType.EAGER)
    @CollectionTable(
        name = "attributed2s",
        joinColumns = @JoinColumn(name = "classa_id"),
        uniqueConstraints = { @UniqueConstraint(columnNames = { "attributed2" }) }
    )
    @Column(name = "attributed2", nullable = false)
    private List<String> attributeD2 = new ArrayList<>();
}
```

Plik: DataTypeA.java

```
package pl.edu.pwr.abis.domain.basic;
```

```
import jakarta.persistence.Basic;
import jakarta.persistence.Embeddable;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Embeddable
public class DataTypeA {

   @Basic
   private Character attributeA;

   @Basic
   private String attributeB;
}
```

Plik: EnumerationA.java

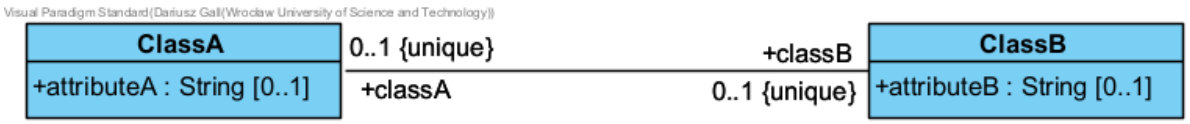
```
package pl.edu.pwr.abis.domain.basic;

public enum EnumerationA {
   LiteralA,
   LiteralB,
}
```

Odwzorowania asocjacji 1-1

Przedstawiono odwzorowania asocjacji 1-do-1. W dalszej części znajdują się przykłady konstrukcji analitycznych i ich odwzorowanie do kodu w Javie + Hibernate.

Przykład 1

PIM



PSM - Implementacja

Plik: ClassA.java

```
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.*;
import lombok.Getter;
```

```java
import lombok.Setter;

@Entity(name = "association_classa")
@Getter
@Setter
public class ClassA {

   @Id
   private Long id;

   @Basic
   private String attributeA;

   @OneToOne(optional = true, mappedBy = "classA")
   private ClassB classB;
}
```

Plik: ClassB.java

```java
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.Basic;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classb")
@Getter
@Setter
public class ClassB {

   @Id
   private Long id;

   @Basic
   private String attributeB;

   @OneToOne(optional = true)
   private ClassA classA;
}
```

Przykład 2

PIM

PSM - Implementacja

Plik: ClassA1.java

```java
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.Basic;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classa1")
@Getter
@Setter
public class ClassA1 {

   @Id
   private Long id;

   @Basic
   private String attributeA;

   @OneToOne(optional = false, mappedBy = "classA1")
   private ClassB1 classB1;
}
```

Plik: ClassB1.java

```java
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.Basic;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classb1")
@Getter
@Setter
public class ClassB1 {

   @Id
   private Long id;

   @Basic
   private String attributeB;

   @OneToOne(optional = true)
   private ClassA1 classA1;
}
```

Odwzorowania asocjacji 1-*

Przedstawiono odwzorowania asocjacji 1-do-wiele. W dalszej części znajdują się przykłady konstrukcji analitycznych i ich odwzorowanie do kodu w Javie + Hibernate.

Przykład 1

PIM

PSM - Implementacja

Plik: ClassA2.java

```
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.*;
import java.util.Set;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classa2")
@Getter
@Setter
public class ClassA2 {

   @Id
   private Long id;

   @Basic
   private String attributeA;

   @OneToMany(mappedBy = "classA2")
   private Set<ClassB2> classB2;
}
```

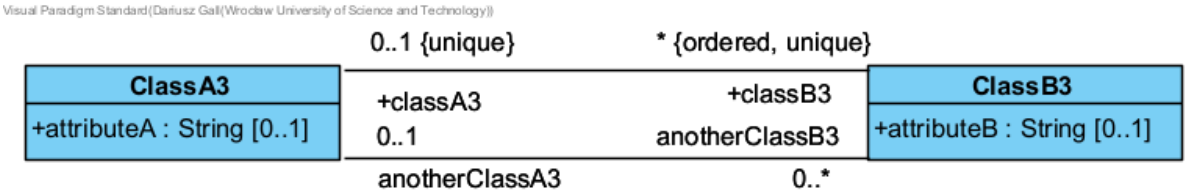
Plik: ClassB2.java

```
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classb2")
@Getter
```

```java
@Setter
public class ClassB2 {

   @Id
   private Long id;

   @Basic
   private String attributeB;

   @ManyToOne(optional = true)
   private ClassA2 classA2;
}
```

Przykład 2

PIM



PSM - Implementacja

Plik: ClassA3.java

```java
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.*;
import java.util.List;

import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classa3")
@Getter
@Setter
public class ClassA3 {

   @Id
   private Long id;

   @Basic
   private String attributeA;

   @OneToMany(mappedBy = "classA3")
   @OrderBy("id")
   private List<ClassB3> classB3;
```

```
    @OneToMany(mappedBy = "otherClassA3")
    private List<ClassB3> otherClassB3;
}
```

Plik: ClassB3.java

```
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.Basic;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classb3")
@Getter
@Setter
public class ClassB3 {

    @Id
    private Long id;

    @Basic
    private String attributeB;

    @ManyToOne(optional = true)
    private ClassA3 classA3;

    @ManyToOne(optional = true)
    private ClassA3 otherClassA3;
}
```

Przykład 3

PIM

PSM - Implementacja

Plik: ClassA4.java

```
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.*;
import java.util.Set;
```

```java
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classa4")
@Getter
@Setter
public class ClassA4 {

   @Id
   private Long id;

   @Basic
   private String attributeA;

   // ewentualnie @ElementCollection ale tylko skierowane asocjacje
   @OneToMany(mappedBy = "classA4", cascade = CascadeType.REMOVE)
   private Set<ClassB4> classB4;
}
```

Plik: ClassB4.java

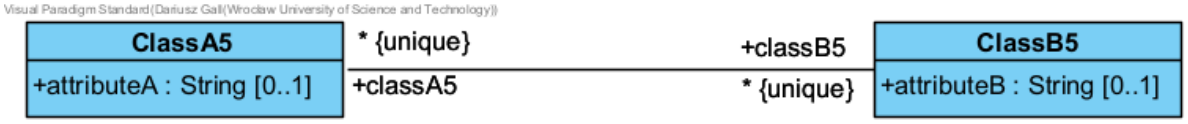
```java
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.Basic;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classb4")
@Getter
@Setter
public class ClassB4 {

   @Id
   private Long id;

   @Basic
   private String attributeB;

   @ManyToOne(optional = true)
   private ClassA4 classA4;
}
```

Odwzorowania asocjacji *-*

Przedstawiono odwzorowania asocjacji wiele-do-wiele. W dalszej części znajdują się przykłady konstrukcji analitycznych i ich odwzorowanie do kodu w Javie + Hibernate.

Przykład 1

PIM



PSM - Implementacja

Plik: ClassA5.java

```
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.*;

import java.util.Set;

import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classa5")
@Getter
@Setter
public class ClassA5 {

   @Id
   private Long id;

   @Basic
   private String attributeA;

   @ManyToMany
   private Set<ClassB5> classB5;
}
```

Plik: ClassB5.java

```
package pl.edu.pwr.abis.domain.association;

import jakarta.persistence.*;
import java.util.Set;
import lombok.Getter;
import lombok.Setter;

@Entity(name = "association_classb5")
@Getter
@Setter
public class ClassB5 {

   @Id
```

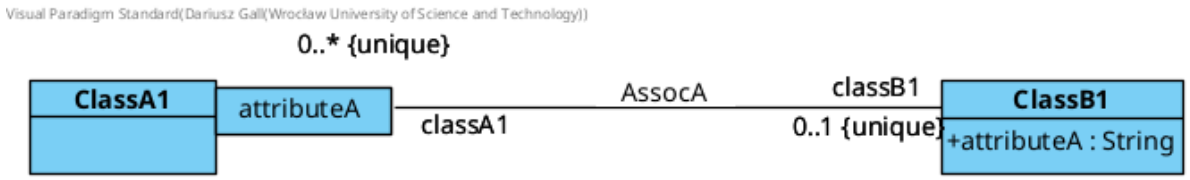
```
    private Long id;

    @Basic
    private String attributeB;

    @ManyToMany(mappedBy = "classB5")
    private Set<ClassA5> classA5;
}
```

Odwzorowania asocjacji z kwalifikatorem

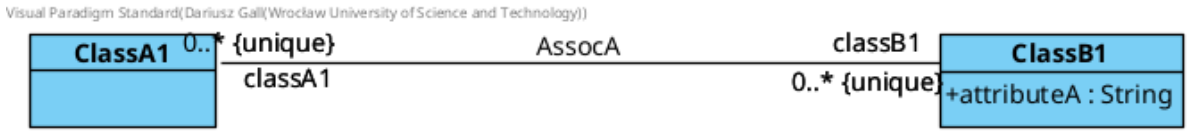
Przedstawiono odwzorowania asocjacji wiele-do-1, 1-do-1, w przypadku gdy po stronie wiele oraz jeden zdefiniowany jest kwalifikator.
W dalszej części znajdują przykłady konstrukcji analitycznych i ich odwzorowanie, inaczej niż dotychczas, do konstrukcji analitycznych, asocjacji, omawianych w poprzednich rozdziałach. Aby w pełni zastosować odwzorowanie, należy odnaleźć wskazane docelowe konstrukcje analityczne i następnie dla każdej z nich dokonać ostatecznej transformacji do kody w Javie + Hibernate.

Przykład 1

PIM



PIM - odwzorowanie



Przykład 2

PIM

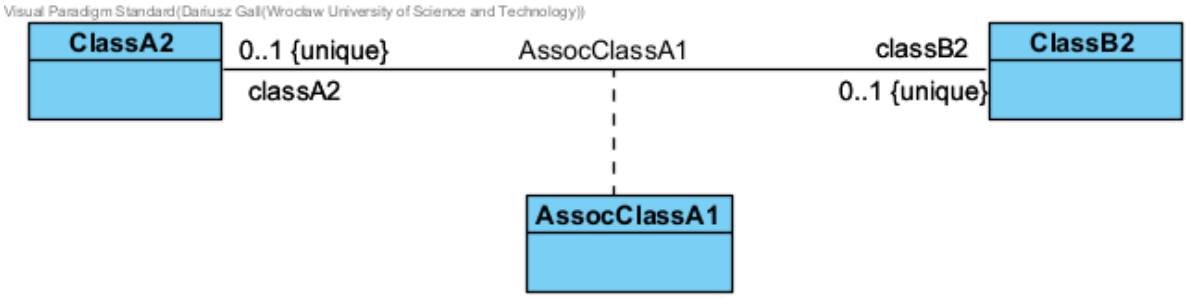
PIM - odwzorowanie

Odwzorowania klas asocjacyjnych 1-1

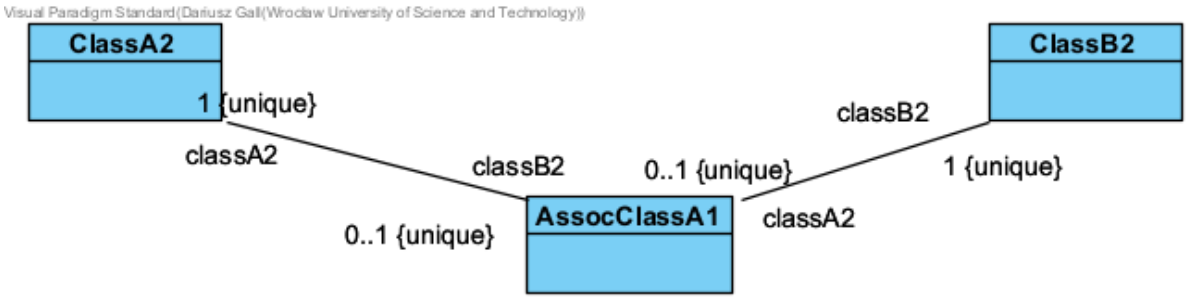
Przedstawiono odwzorowania klas asocjacyjnych 1-do-1.
W dalszej części znajdują przykłady konstrukcji analitycznych i ich odwzorowanie, inaczej niż dotychczas, do konstrukcji analitycznych, asocjacji, omawianych w poprzednich rozdziałach. Aby w pełni zastosować odwzorowanie, należy odnaleźć wskazane docelowe konstrukcje analityczne i następnie dla każdej z nich dokonać ostatecznej transformacji do kody w Javie + Hibernate.

Przykład 1

PIM

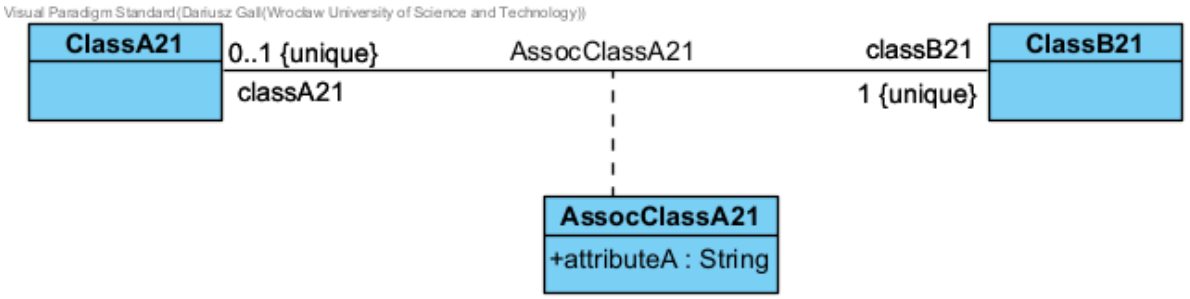


PIM - odwzorowanie

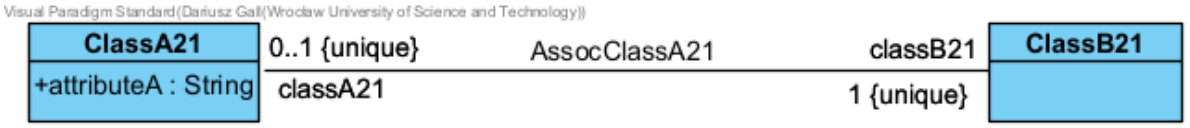


Przykład 2

PIM

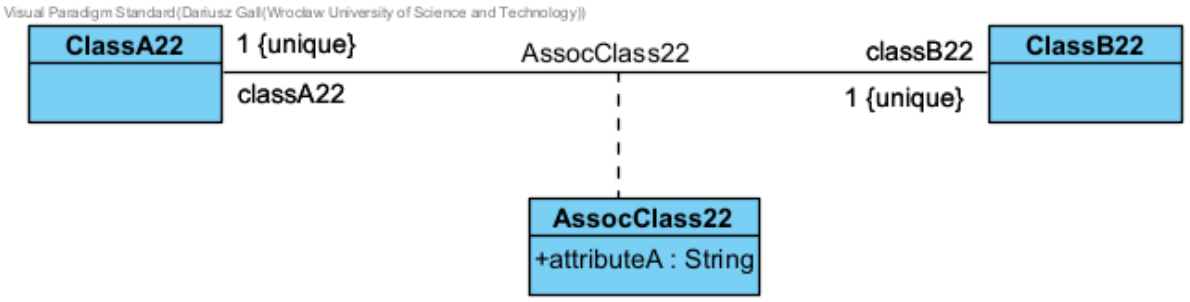


PIM - odwzorowanie

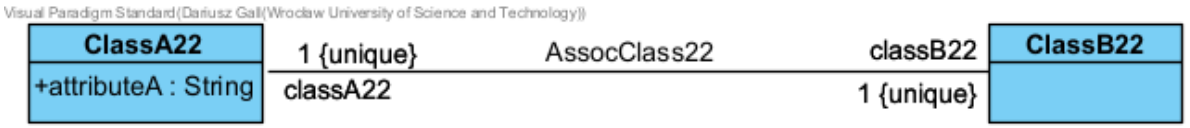


Przykład 3

PIM



PIM - odwzorowanie



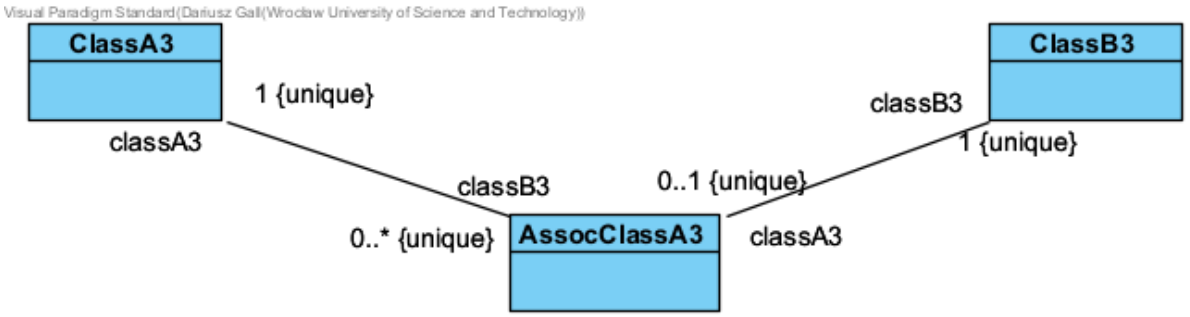
Odwzorowania klas asocjacyjnych 1-*

Przedstawiono odwzorowania klas asocjacyjnych 1-do-wiele.
W dalszej części znajdują przykłady konstrukcji analitycznych i ich odwzorowanie, inaczej niż dotychczas, do konstrukcji analitycznych, asocjacji, omawianych w poprzednich rozdziałach. Aby w pełni zastosować odwzorowanie, należy odnaleźć wskazane docelowe konstrukcje analityczne i następnie dla każdej z nich dokonać ostatecznej transformacji do kody w Javie + Hibernate.

Przykład 1

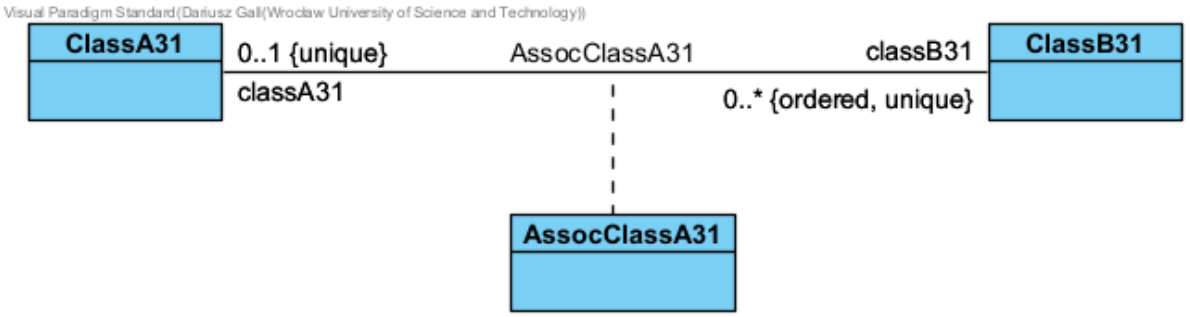
PIM

PIM - odwzorowanie

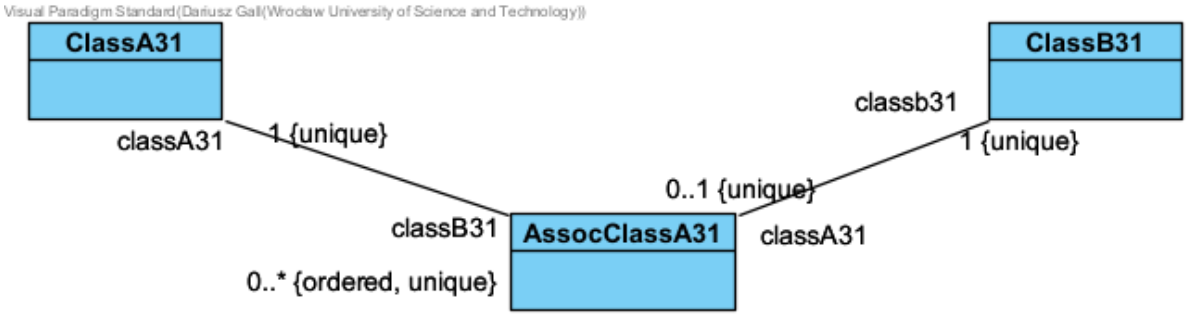


Przykład 2

PIM



PIM - odwzorowanie



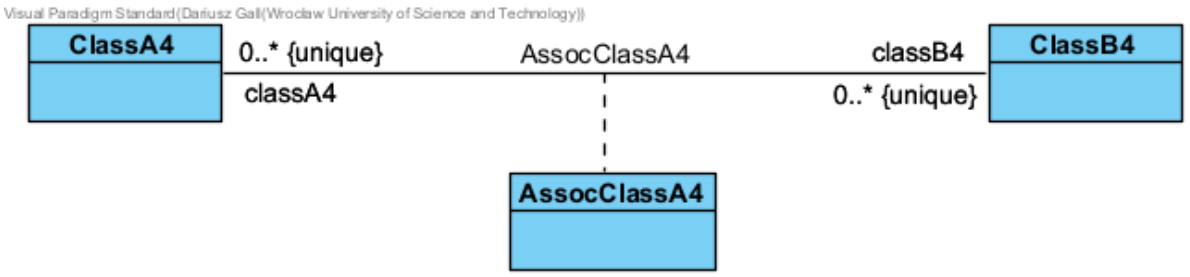
Odwzorowania klas asocjacyjnych *-*

Przedstawiono odwzorowania klas asocjacyjnych wiele-do-wiele.
W dalszej części znajdują przykłady konstrukcji analitycznych i ich odwzorowanie, inaczej niż dotychczas, do konstrukcji analitycznych, asocjacji, omawianych w poprzednich rozdziałach. Aby w pełni zastosować odwzorowanie, należy odnaleźć wskazane docelowe

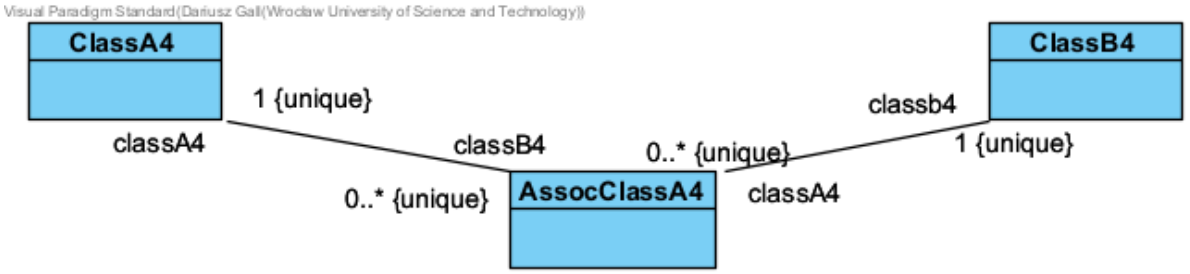
konstrukcje analityczne i następnie dla każdej z nich dokonać ostatecznej transformacji do kody w Javie + Hibernate.

Przykład 1

PIM

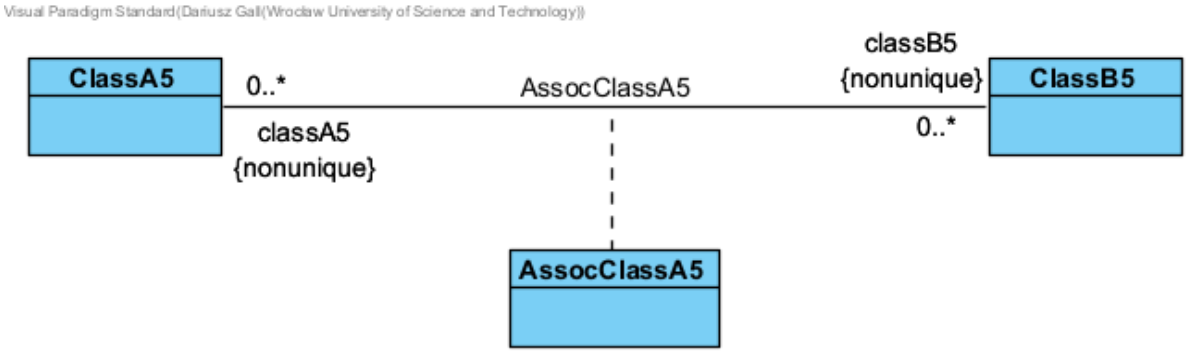


PIM - odwzorowanie

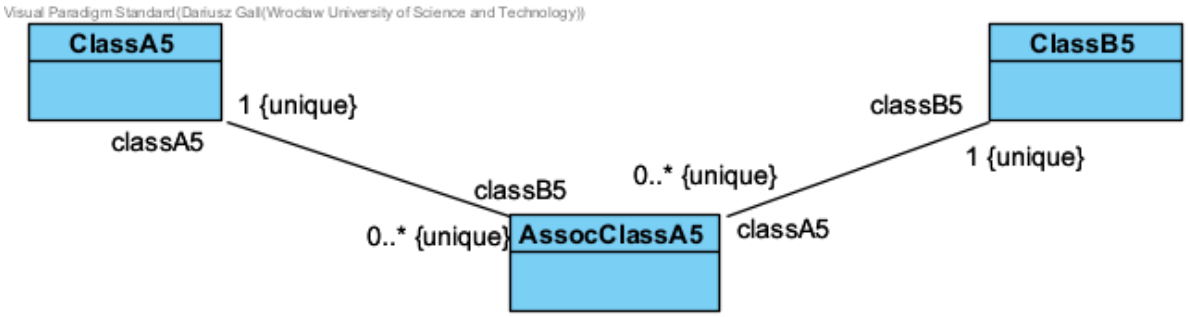


Przykład 2

PIM



PIM - odwzorowanie



Odwzorowania relacji generalizacji.

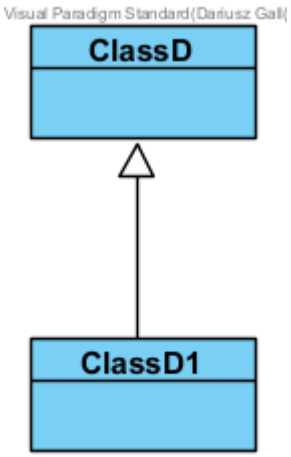
Przedstawiono odwzorowania relacji generalizacji w dla przypadków:

- dziedziczenia pojedynczej klasy z pojedynczej klasy, dla dowolnych zbiorów generalizacji (brak wielodziedziczenia)
- dziedziczenia z pojedynczej klasy, dla zbiorów generalizacji {complete, disjoint}, {incomplete, disjont} (brak wieloinstancyjność, brak wielodziedziczenia)

W dalszej części znajdują się przykłady konstrukcji analitycznych i ich odwzorowanie do kodu w Javie + Hibernate.

Przykład 1

PIM



PSM - Implementacja

Plik: ClassD.java

```
package pl.edu.pwr.abis.domain.generalization;
```

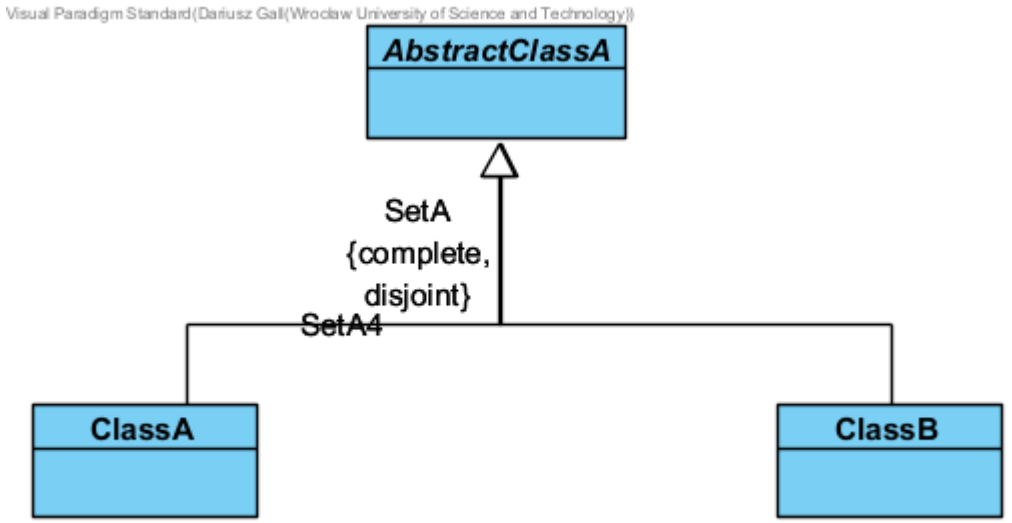
```java
import jakarta.persistence.DiscriminatorColumn;
import jakarta.persistence.DiscriminatorType;
import jakarta.persistence.Id;
import jakarta.persistence.Inheritance;
import jakarta.persistence.InheritanceType;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="type",
   discriminatorType = DiscriminatorType.STRING)
public class ClassD {

   @Id
   private Long id;
}
```

Plik: ClassD1.java

```java
package pl.edu.pwr.abis.domain.generalization;

import jakarta.persistence.DiscriminatorValue;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@DiscriminatorValue("ClassD1")
public class ClassD1 extends ClassD {}
```

Przykład 2

PIM



PSM - Implementacja

Plik: AbstractClassA.java

```
package pl.edu.pwr.abis.domain.generalization;

import jakarta.persistence.DiscriminatorColumn;
import jakarta.persistence.DiscriminatorType;
import jakarta.persistence.Id;
import jakarta.persistence.Inheritance;
import jakarta.persistence.InheritanceType;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="type",
   discriminatorType = DiscriminatorType.STRING)
public abstract class AbstractClassA {
   @Id
   private Long id;
}
```

Plik: ClassA.java

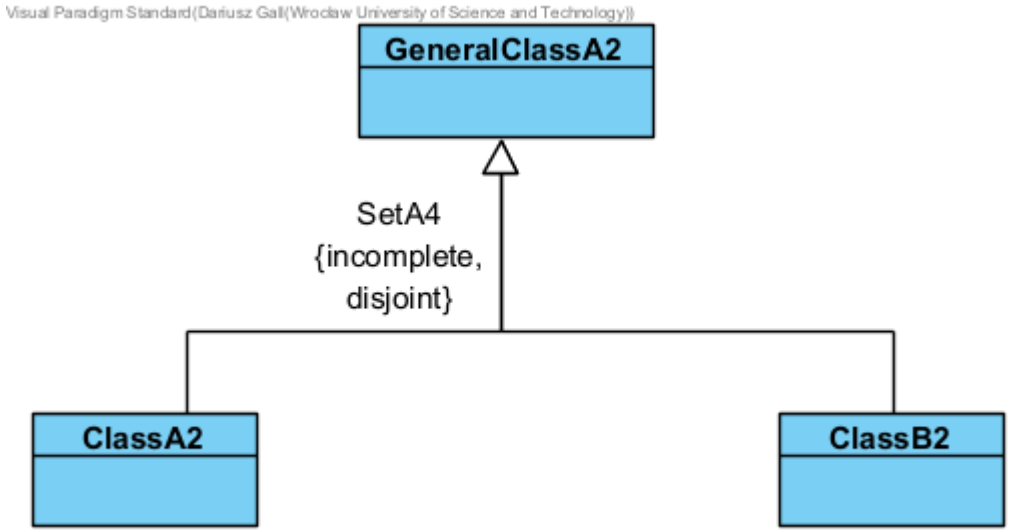
```
package pl.edu.pwr.abis.domain.generalization;

import jakarta.persistence.DiscriminatorValue;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@DiscriminatorValue("ClassA")
public class ClassA extends AbstractClassA {
}
```

Plik: ClassB.java

```
package pl.edu.pwr.abis.domain.generalization;

import jakarta.persistence.DiscriminatorValue;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@DiscriminatorValue("ClassB")
public class ClassB extends AbstractClassA {
}
```

Przykład 3

PIM



PSM - Implementacja

Plik: GeneralClassA2.java

```
package pl.edu.pwr.abis.domain.generalization;

import jakarta.persistence.DiscriminatorColumn;
import jakarta.persistence.DiscriminatorType;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Inheritance;
import jakarta.persistence.InheritanceType;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Entity(name = "generalization-generalclassa2")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="type",
   discriminatorType = DiscriminatorType.STRING)
public class GeneralClassA2 {

   @Id
   private Long id;
}
```

Plik: ClassA2.java

```
package pl.edu.pwr.abis.domain.generalization;
```

```
import jakarta.persistence.DiscriminatorValue;

@DiscriminatorValue("ClassA2")
public class ClassA2 extends GeneralClassA2 {}
```

Plik: ClassB2.java

```
package pl.edu.pwr.abis.domain.generalization;

import jakarta.persistence.DiscriminatorValue;

@DiscriminatorValue("ClassB2")
public class ClassB2 extends GeneralClassA2 {}
```

Odwzorowania relacji generalizacji. Wielodziedziczenia, wieloinstancyjność.

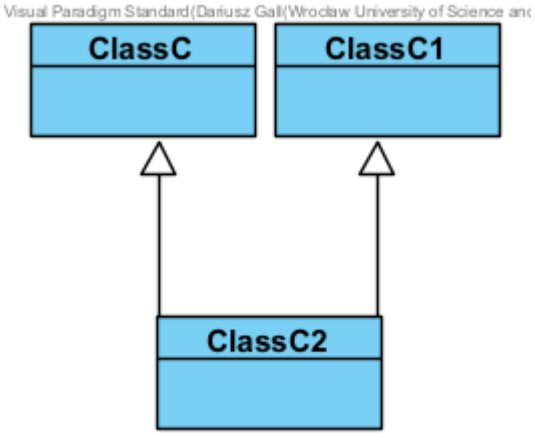
Przedstawiono odwzorowania relacji generalizacji w dla przypadków:

- dziedziczenia pojedynczej klasy z wielu klasy, dla dowolnych zbiorów generalizacji (wielodziedziczenie)
- dziedziczenia z pojedynczej klasy, dla zbiorów generalizacji {complete, overlapping}, {incomplete, overlapping} (wieloinstancyjność, brak wielodziedziczenia)

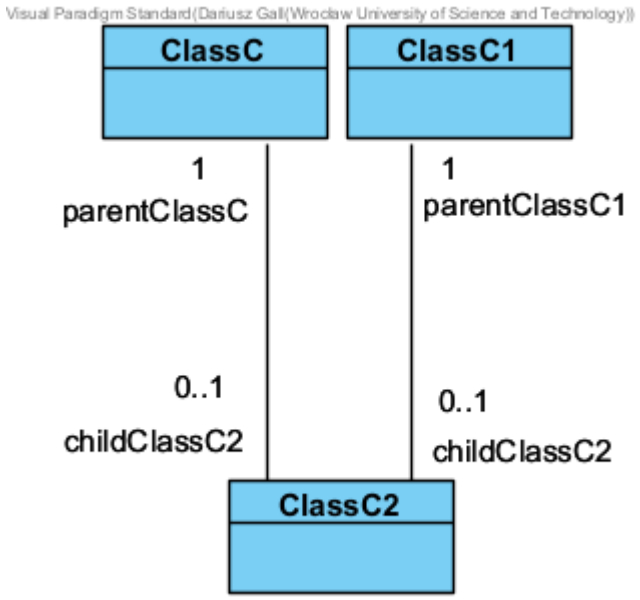
W dalszej części znajdują przykłady konstrukcji analitycznych i ich odwzorowanie, inaczej niż dotychczas, do konstrukcji analitycznych, asocjacji, omawianych w poprzednich rozdziałach. Aby w pełni zastosować odwzorowanie, należy odnaleźć wskazane docelowe konstrukcje analityczne i następnie dla każdej z nich dokonać ostatecznej transformacji do kody w Javie + Hibernate.

Przykład 1

PIM

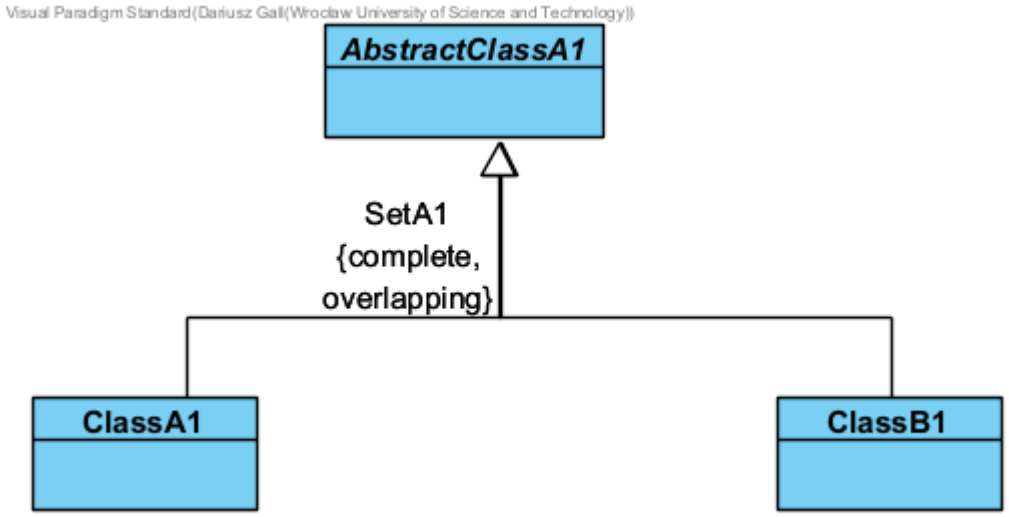


PIM - odwzorowanie

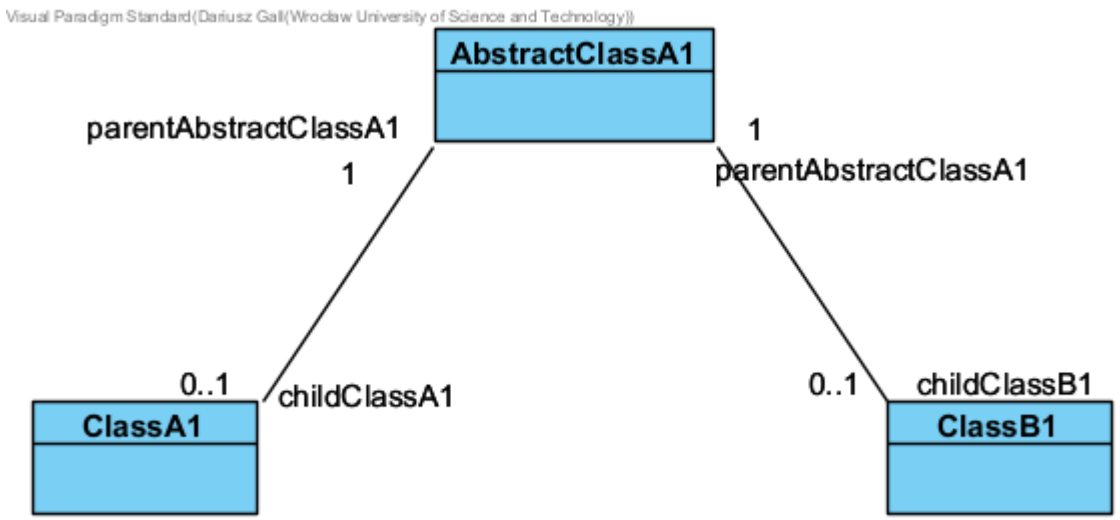


Przykład 2

PIM

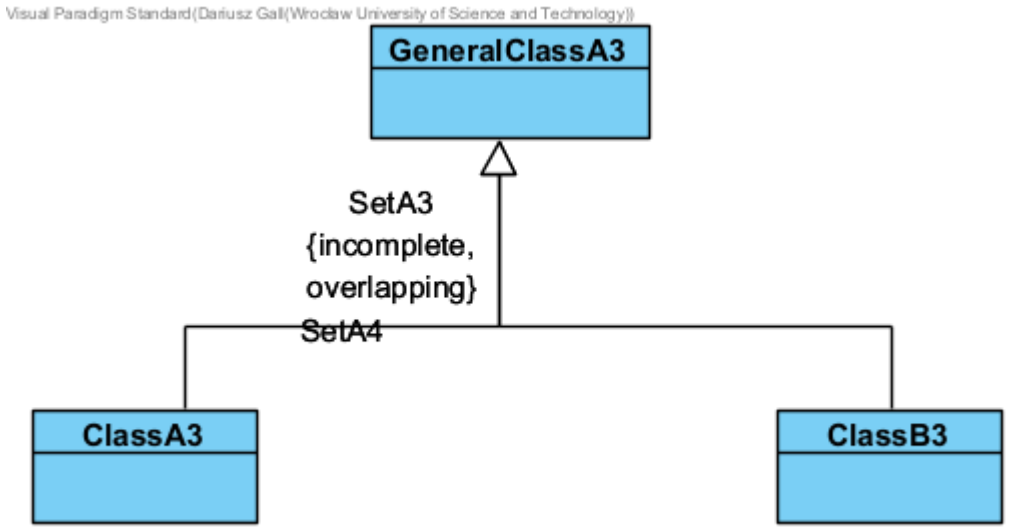
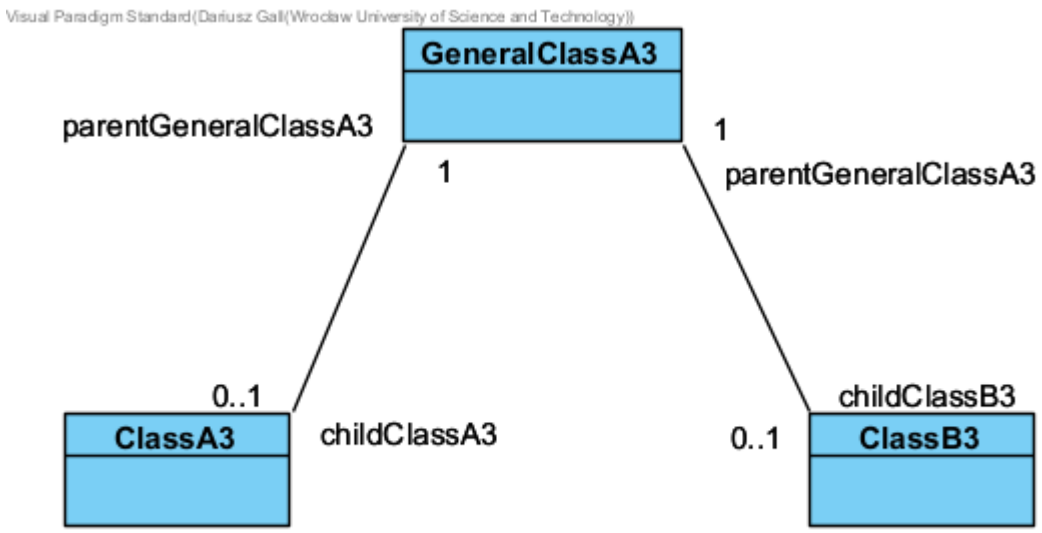


PIM - odwzorowanie



Przykład 3

PIM



PIM - odwzorowanie